

SO2 - Etapa 4

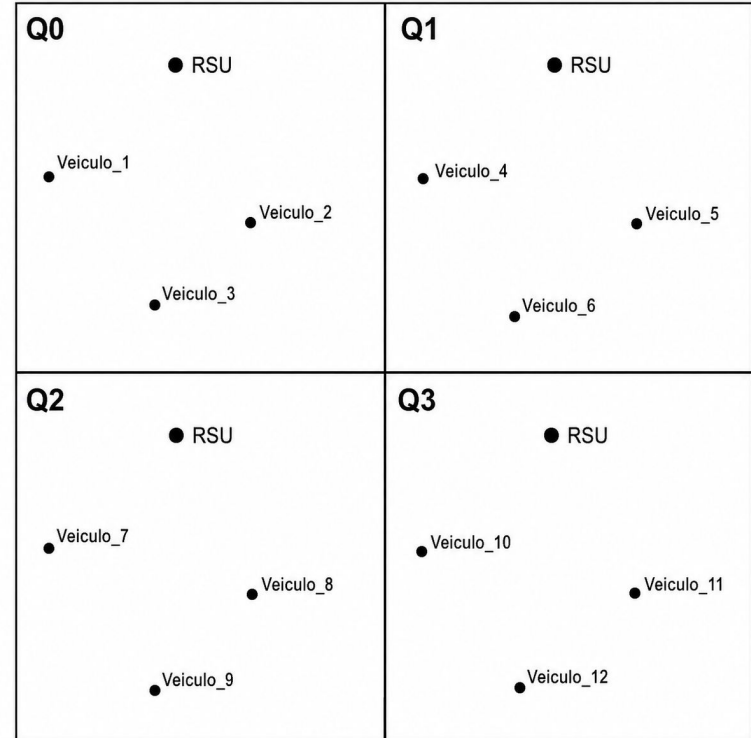
Sincronização Espacial entre Sistemas Autônomos

Integrantes do Grupo

Arthur Erpen, Caetano Peruzzo e Gabriel Herrera

Objetivos:

- 1) Mensagens etiquetadas com coordenadas espaciais
- 2) Precisão de quadrante
- 3) Componentes no mesmo sistema compartilham percepção
- 4) Módulo de kernel com interface de leitura
- 5) Coordenadas inicialmente aleatórias
- 6) $M = \{\text{origin, timestamp, payload}\}$
- 7) Formato PTP preservado



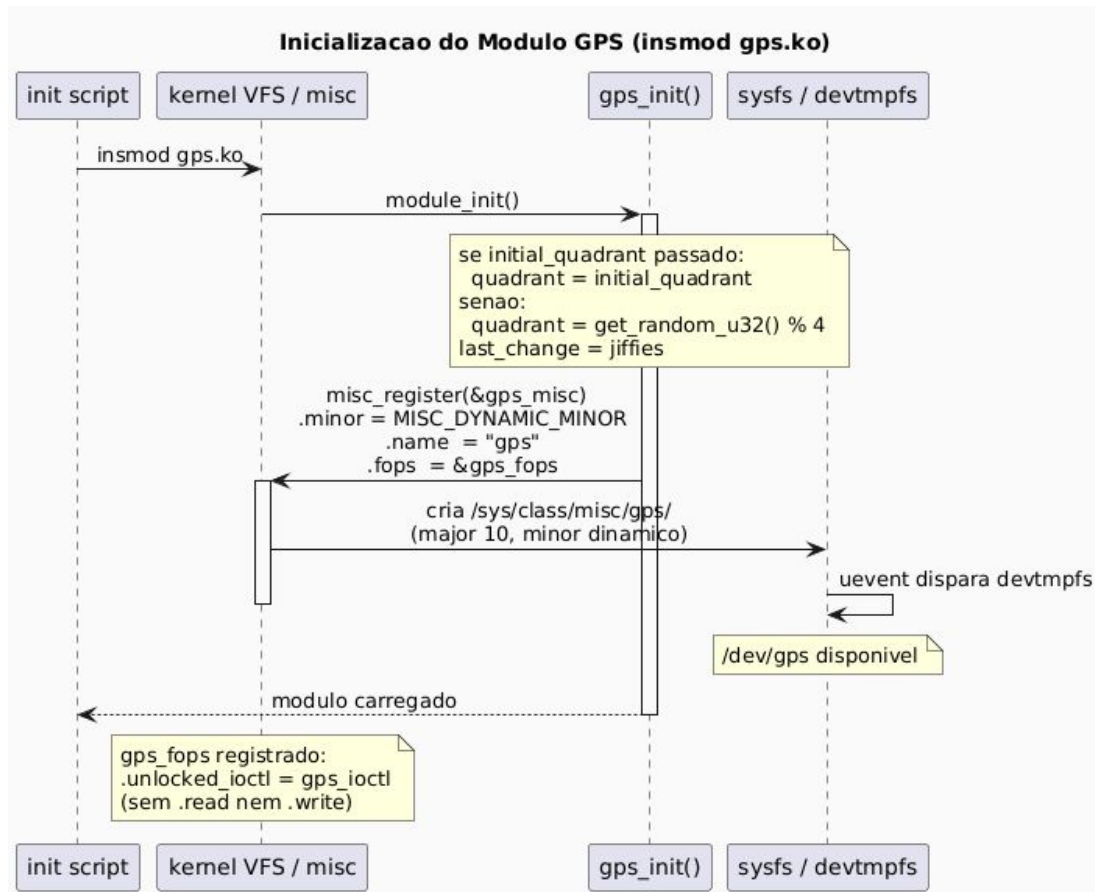
Header do Protocol:

- Agora a informação de qual quadrante uma mensagem se originou é carregada no header do nível de protocolo

```
1 class Header {  
2     // ...  
3  
4     private:  
5         Port _src_port;  
6         Port _dst_port;  
7         uint8_t _quadrant;  
8         PacketKind _kind;  
9 };
```

Módulo de Kernel GPS

- Misc device (major 10 reservado pelo kernel na nossa distribuição linux)
- Interface exclusivamente via ioctl (sem read/write)
- Todos processos da VM enxergam o mesmo quadrante



Lado Kernel

Mundo do kernel (espaço privilegiado)



gps.c

Toda a lógica do GPS:
estado do quadrante, avanço
preguiçoso, handler de ioctl.
Vira o **gps.ko**.



gps_ioctl.h

Contrato compartilhado.
Define os números dos ioctls.
Único arquivo incluído pelos dois mundos.

Lado Userspace

Mundo da aplicação (espaço não privilegiado)



gps.h

Declaração da classe C++ GPS.
Interface amigável pra aplicação.

</> gps.cpp

Implementação da classe.
Encapsula as chamadas a `/dev/gps`
e devolve valores tratados.



Runtime — /dev/gps

Canal de comunicação entre os mundos.
Kernel expõe via **gps.ko**; userspace consome via **classe GPS**.

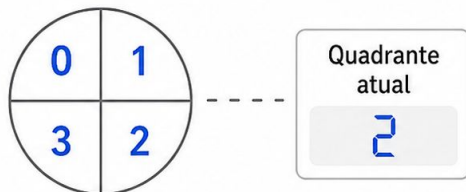


Resumo: Cada arquivo tem um papel distinto. **gps.c** é o cérebro, **gps_ioctl.h** é a ponte, **gps.h** é a fachada, **gps.cpp** é o tradutor.

Interface ioctl

GPS_IOC_GET_QUADRANT

Retorna quadrante atual (0–3)



Dispara avanço preguiçoso



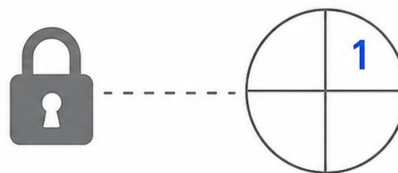
Usa **jiffies** como contador



jiffies
123456

GPS_IOC_SET_FIXE D

Congela o quadrante



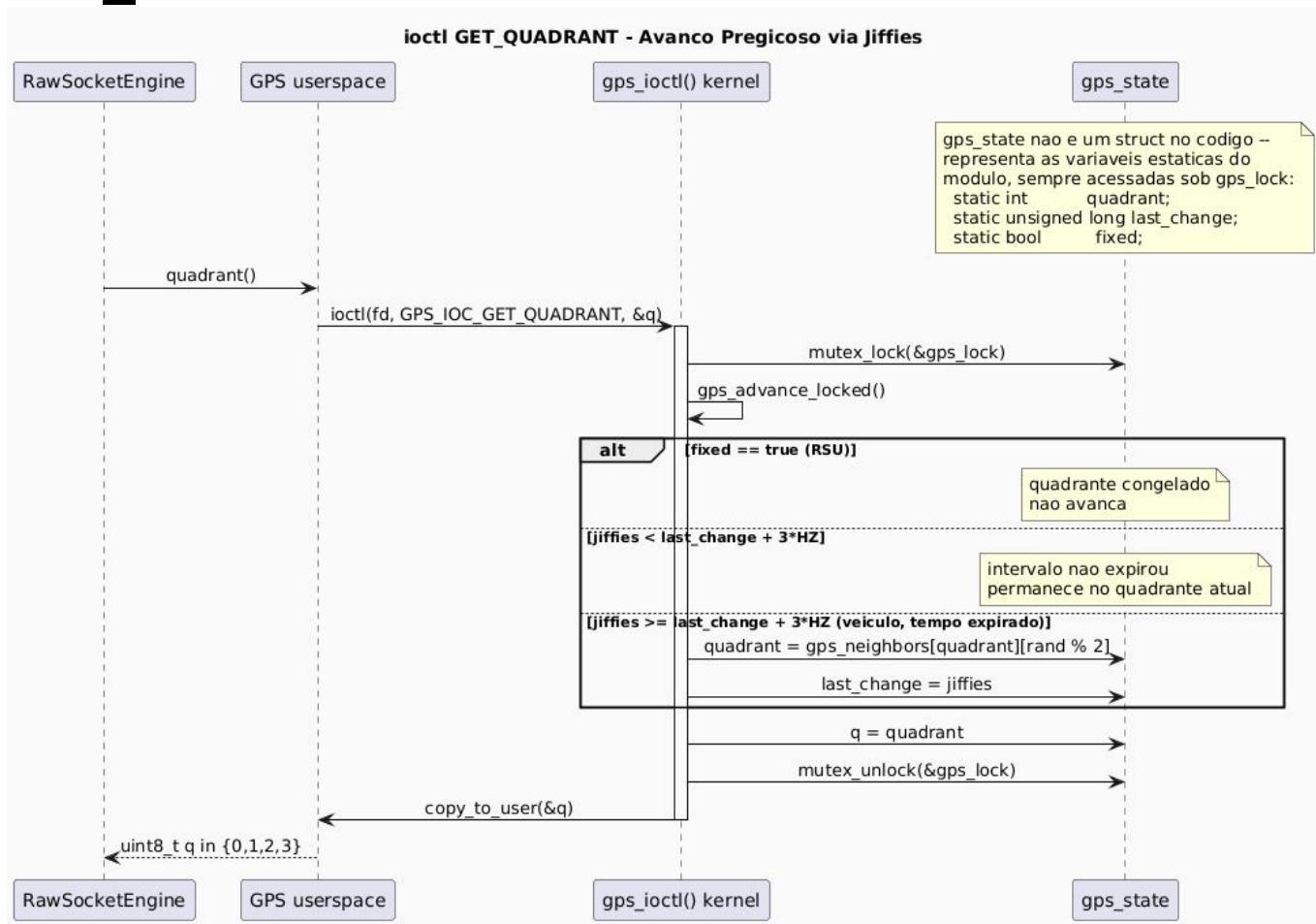
Chamado **1×** na RSU



Veículo nunca chama

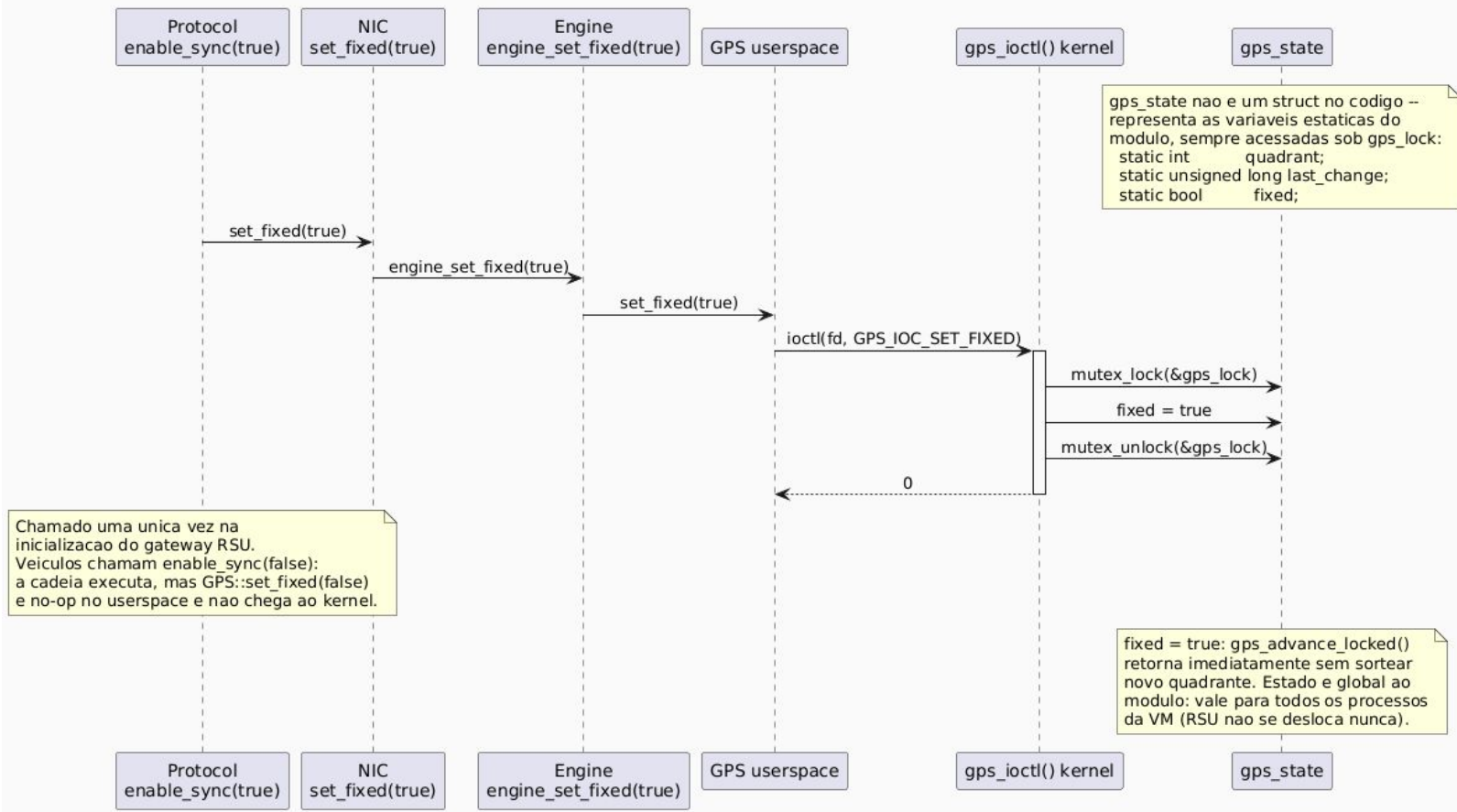


Fluxo GET_QUADRANT



Fluxo SET_FIXED

RSU - Congelamento de Quadrante via SET_FIXED



Avanço Preguiçoso e Adjacência



1) Sem timer/thread

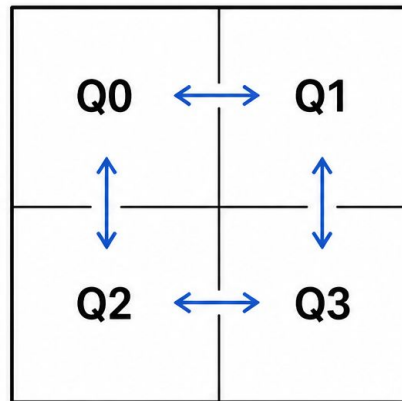
O sorteio acontece dentro do `ioctl`,
comparando `jiffies` com
`last_change + 3*HZ`



2) Apenas quadrantes adjacentes

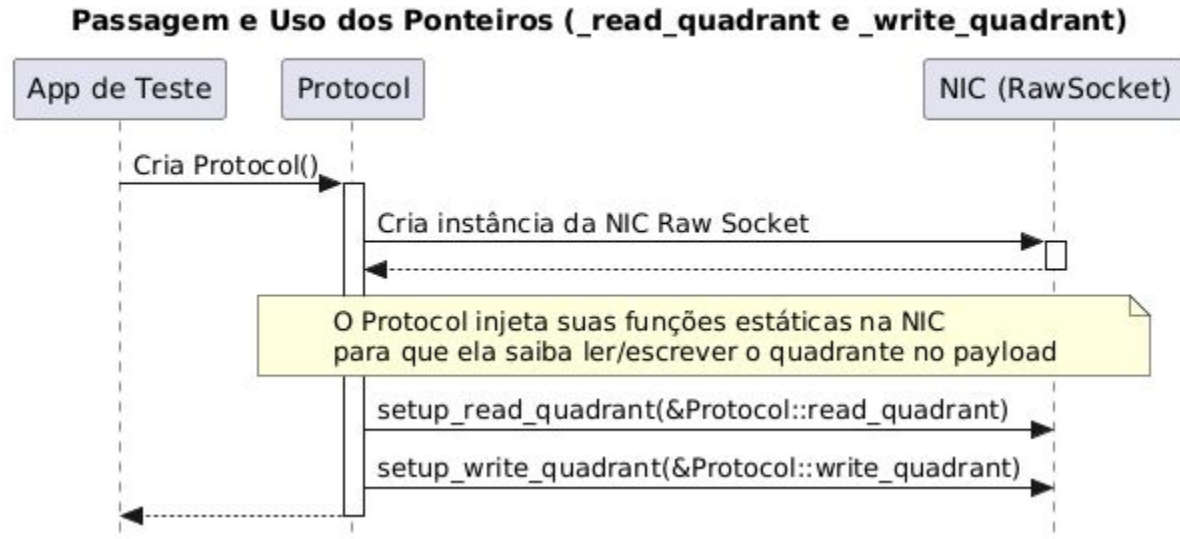
Tabela de adjacência:
`gps_neighbors[4][2]`

Quadrante	Vizinhos (adjacentes)
Q0	Q1, Q2
Q1	Q0, Q3
Q2	Q0, Q3
Q3	Q1, Q2



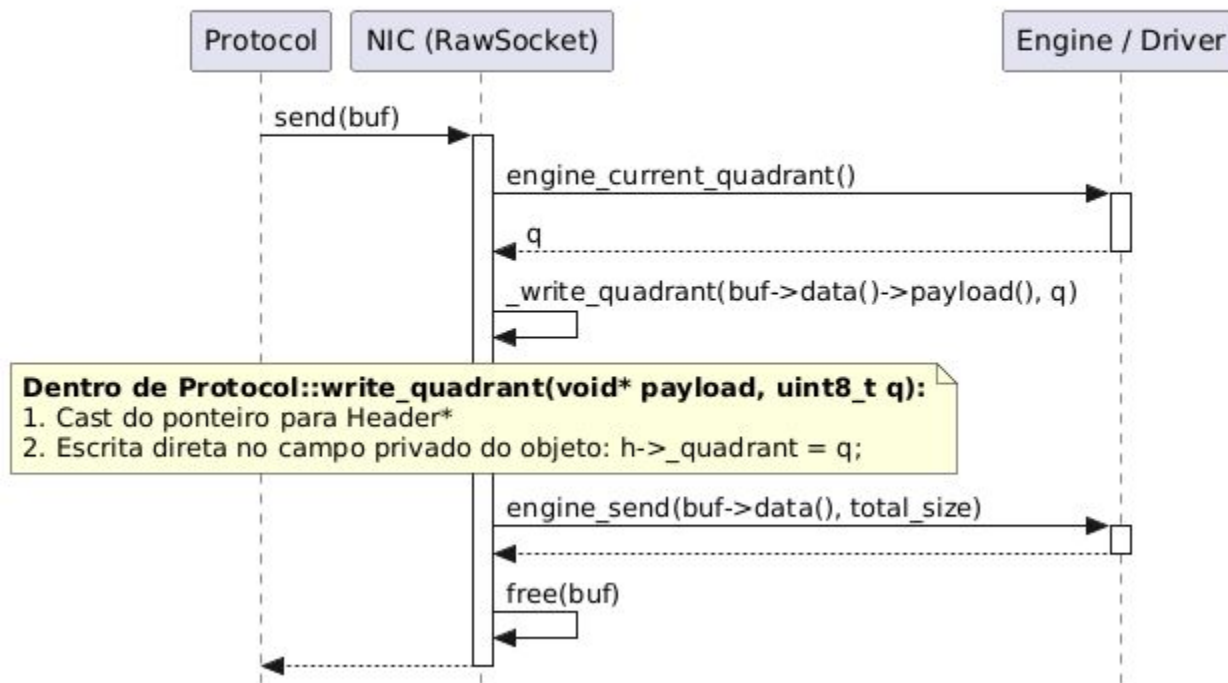
Q0 pode ir para Q1 ou Q2.
Nunca para Q3.

Read e Write Quadrant na NIC



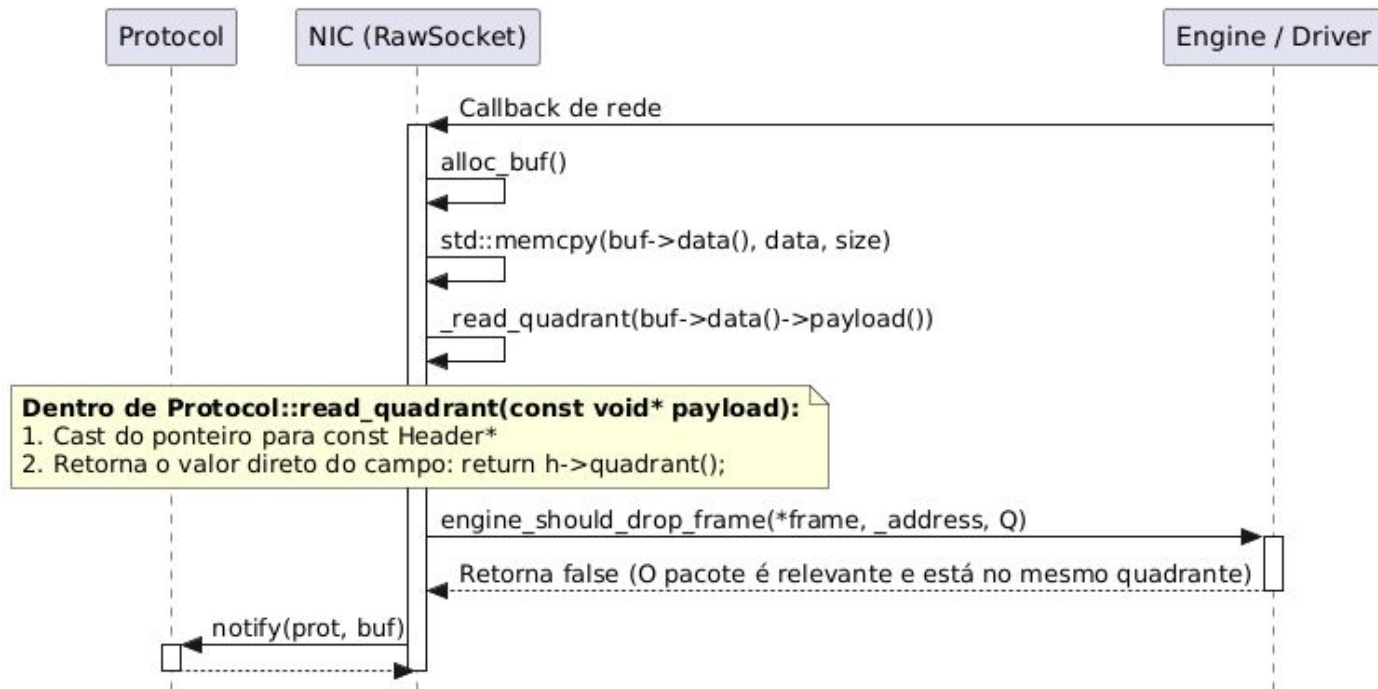
Fluxo de Envio

Fluxo de Envio - Escrita no Header via `_write_quadrant`



Fluxo de Recebimento

Fluxo de Recebimento - Leitura do Header via `_read_quadrant`



Teste Sincronização Espacial

- 9 VMs rodando, sendo as 4 primeiras RSUs (uma por quadrante) e as demais veículos.
- Os veículos se deslocam a cada ≥ 3 s.
- 90s de simulação.

Resultados (por VM):

VM	ROLE	RECEIVED	SAME_QUADRANT	CROSS_QUADRANT	QUAD_CHANGES
 [vm1]	RSU	-	-	-	0
 [vm2]	RSU	-	-	-	0
 [vm3]	RSU	-	-	-	0
 [vm4]	RSU	-	-	-	0
 [vm5]	veiculo	111	111	0	30
 [vm6]	veiculo	112	112	0	30
 [vm7]	veiculo	115	115	0	30
 [vm8]	veiculo	105	105	0	30
 [vm9]	veiculo	39	39	0	30

✓ **quadrant aprovado**



RSU: Road Side Unit (infraestrutura fixa)



veiculo: Veículo em movimento

RECEIVED: Total de mensagens recebidas

SAME_QUADRANT: Mensagens recebidas do mesmo quadrante

CROSS_QUADRANT: Mensagens recebidas de quadrante diferente

QUAD_CHANGES: Total de mudanças de quadrante do veículo

Obrigado!